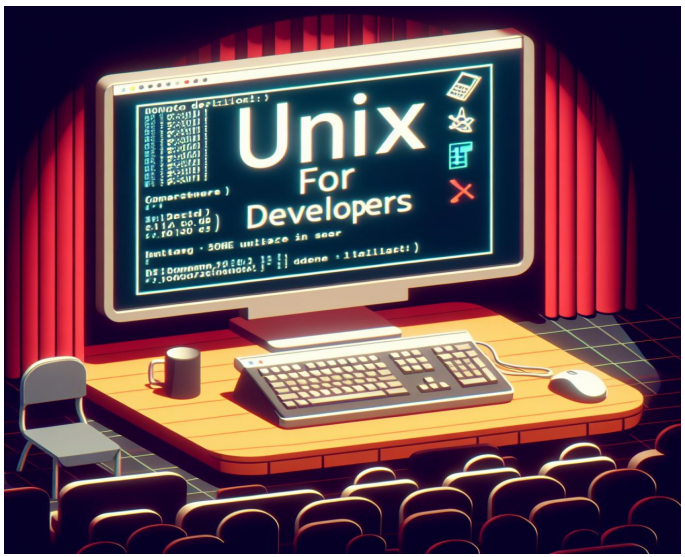
Advanced Shell Programming

Benedict Reuschling

December 2, 2024



Advanced Shell Programming

This section delves into advanced concepts of shell programming, expanding upon the foundational principles covered in the previous section. It elucidates extra functionalities to enhance the power of shell programming. Topics include using here documents to automate input for interactive programs, leveraging cdialog to craft more engaging dialogues and menus for user interaction, understanding how functions operate within shells, and calling them from trapsan influential mechanism during script execution.

Overview

1. Here documents
2. Text-based User Interfaces with `cdialog`
3. Functions
4. Traps

Here Documents

Use lines as input within a shell script for interactive commands. These lines serve as input for the program instead of relying on standard input. This approach has the advantage of allowing the shell script to bypass waiting for user input, which may never occur, and instead receive input directly from the script. These constructs are known as here documents and follow this syntax:

```
  Command <<Delimiter
Input lines for the command
more lines for the command
...
  Delimiter
```

Any string is possible as the `Delimiter`. At the end of the here document the same delimiter terminates the input lines and the shell script continues.
Important: The closing delimiter must be on a line of its own.

Indentation between Delimiters

Normally, no indentation (tabs, spaces) of lines is allowed between the starting and ending delimiter. This makes shell scripts less good to read. Indentation **is** allowed when prefixing the `starting` delimiter (and only that) with a dash:

```
Command <<-Delimiter
 Input lines for the command
 more lines for the command
 ...
Delimiter
```

Usage Example for Here Documents

This example uses bc to make calculations with arbitrary precision, something which is not possible with expr. Since bc is an interactive program the here document provides all input from the shell script.

```
$ cat circumference.sh
#!/bin/sh
PI=3.1415
echo "Please enter circle radius:"
read radius
circumference=$(bc <<EOF
2*$PI*$radius
EOF)
echo "Circumference is $circumference"
$ circumference.sh
$ Please enter circle radius:
5
Circumference is 31.4150
$
```

Chose the delimiter name (EOF here) freely. Exit the here document by reusing the delimiter.

Further Use of Here Documents

Another use case for here documents in shell scripts is inserting text into a file. Until now, doing this has been a bit cumbersome:

```
FILE=/tmp/mylog.txt
echo "A line" >> $FILE
echo "another one" >> $FILE
```

This script is not maintainable, because the file name `FILE` has to appear in more than one line. Here documents simplify the code of the shell script in combination with the `cat` command:

```
FILE=/tmp/mylog.txt
cat << -EOF > $FILE
    A line
    another one
EOF
```

In this example though, the contents of the file are overwritten. When adding text at the end of the file use the following syntax:

```
cat << EOF >> $FILE
    ...
EOF
```

Overview

1. Here documents
2. Text-based User Interfaces with cdialog
3. Functions
4. Traps

Text-based User Interfaces with cdialog

Use the program cdialog[1] to create dialog boxes. These have a similar look and feel as the ones known from graphical user interfaces (GUI). They accept keyboard input, ↵, ⇤⇥, and Space. Entering text via input fields lets users fill variables.
All cdialog boxes must have at least the following three parameters:

`Text:`	The headline or the content of the box
`Height:`	The height of the box in console lines
`Width:`	The width of the box in console lines

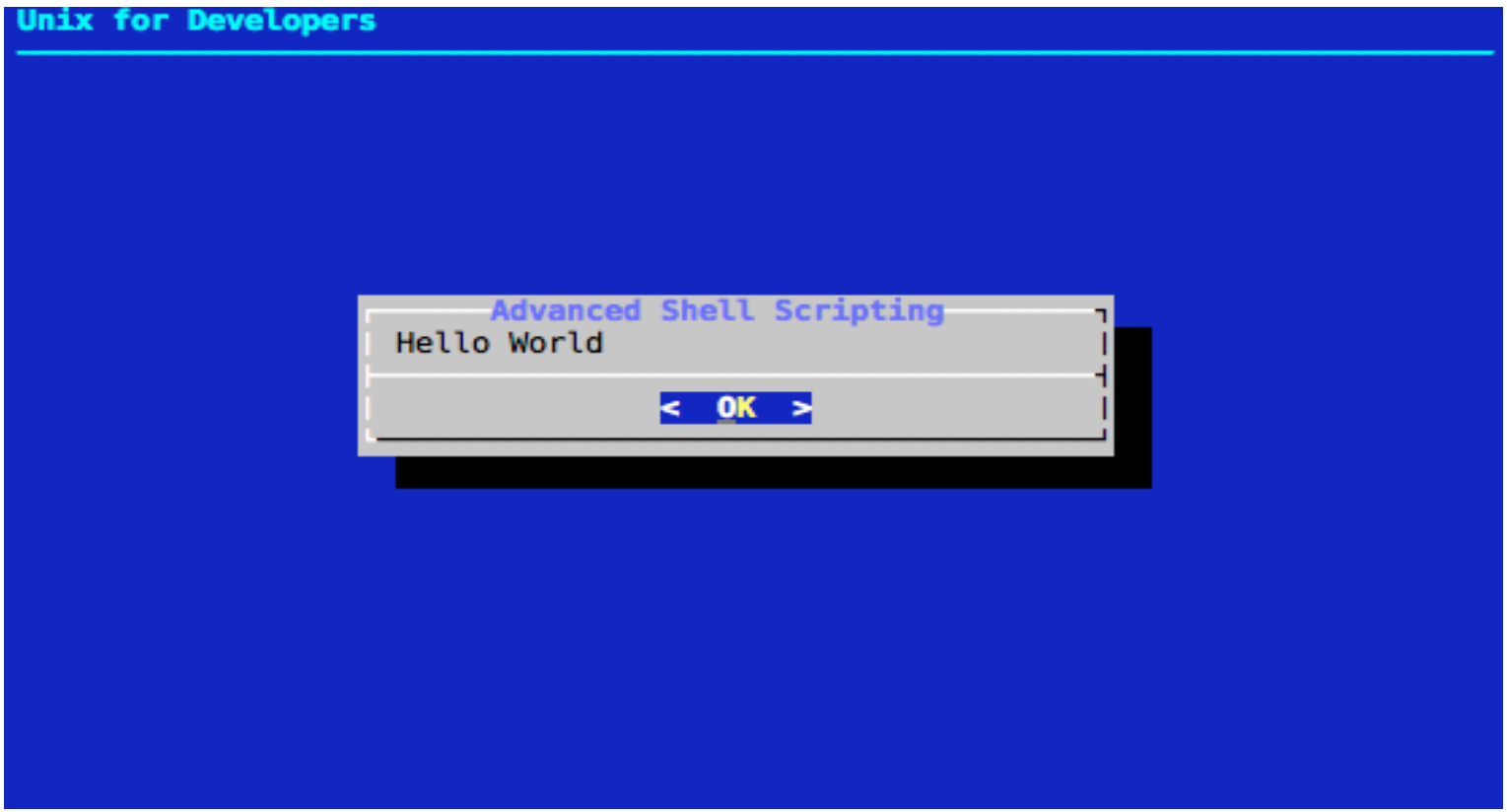
Other parameters depend upon the box type used.

[1] A dialog program is pre-installed in most Unix distributions. If not, install it via the package manager or compile it from source. See http://invisible-island.net/dialog/

Example for a Messagebox with cdialog

The following script is an example for a message box (wrapped parameter lines).

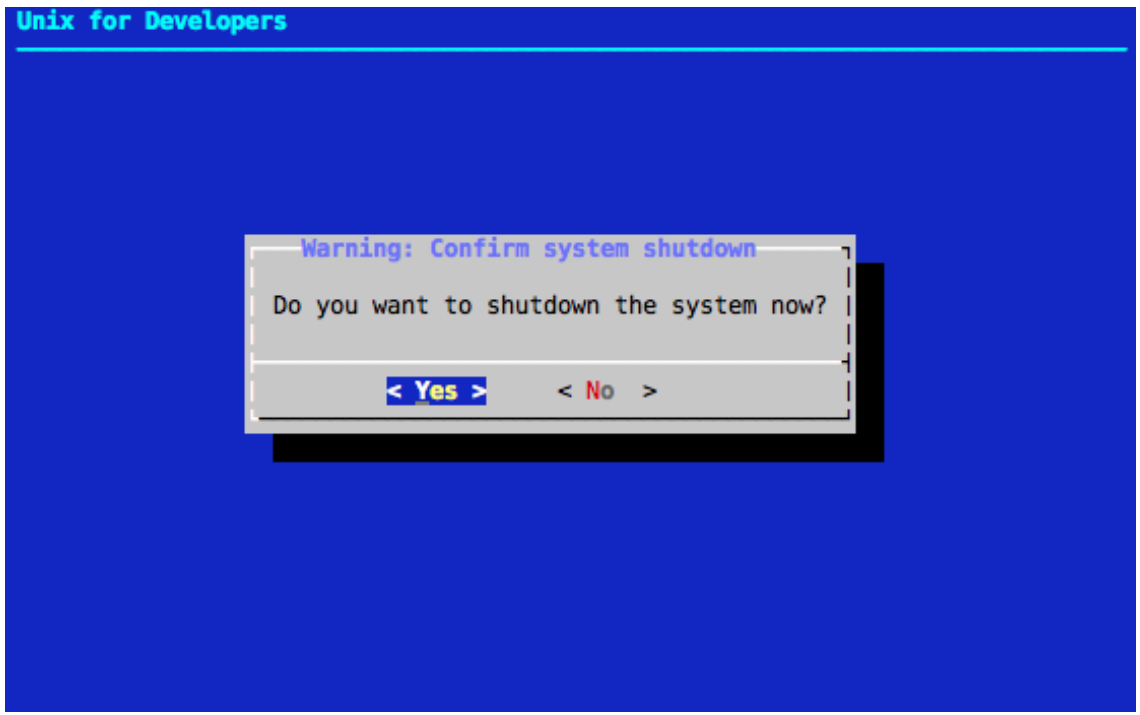
```
$ cdialog --title "Advanced Shell Scripting"
--backtitle "Unix for Developers"
--msgbox "Hello World" 5 40
```



yes/no Input

Often, the user should decide between two options (or cancel using the Esc key). The following cdialog box allows this:

```
$ cdialog --title "Warning: Confirm system shutdown"
--backtitle "Unix for Developers" --yesno
"\nDo you want to shutdown the system now?" 7 43
```



Example Usage for yes/no Boxes

In this example, the variable `selection` stores the result of the previous `cdialog` command using `$?`.

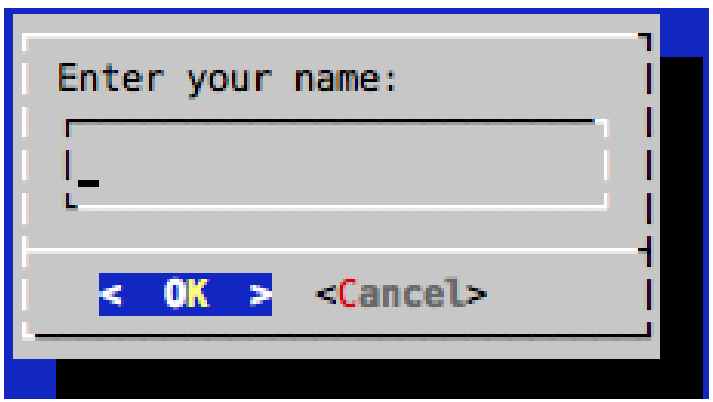
```
$ cat myyesno.sh
#!/bin/sh
cdialog --title "Warning: Confirm system shutdown"
--backtitle "Unix for Developers" --yesno
"\nDo you want to shutdown the system now?" 7 43
selection=$?
case $selection in
    0) echo "Initiating shutdown...";;
    1) echo "Shutdown aborted!";;
    255) echo "User canceled by pressing ESC";;
esac
```

Pressing Esc, `cdialog` returns a special code (255). A `case` statement evaluates it to programmatically react to this keypress.

Input boxes as Replacement for `read`

Inputboxes allow it to get user input in a more visually pleasing way than using `read`.
The option `--inputbox` of `cdialog` also allows predefined values, which the user can choose to change when needed.
Depending which option the user has chosen (OK, Cancel, or Esc) `cdialog` returns another value (`$?`).

- 0: OK selected (does not mean an input happened)
- 1: Cancel selected
- 255: Esc pressed



Example for Using Inputboxes

When incorporating cdialog into shell scripts, especially when not using it directly from the terminal, it's essential to redirect both the standard input and output channels. By default, the output of cdialog appears on standard error (stderr). To redirect the user input from that box into a variable instead of a temporary file, use the following construct:

```
3>&1 1>&2 2>&3
```

- `3>&1`: Opens a file descriptor that redirects stderr to stdout
- `1>&2`: Redirects stdout to stdin
- `2>&3`: Redirects stdin to stderr, closing the loop

This redirects the standard input, standard error and standard output channels, respectively. The result is that the cdialog inputbox appears on the screen.

The following example shows the usage:

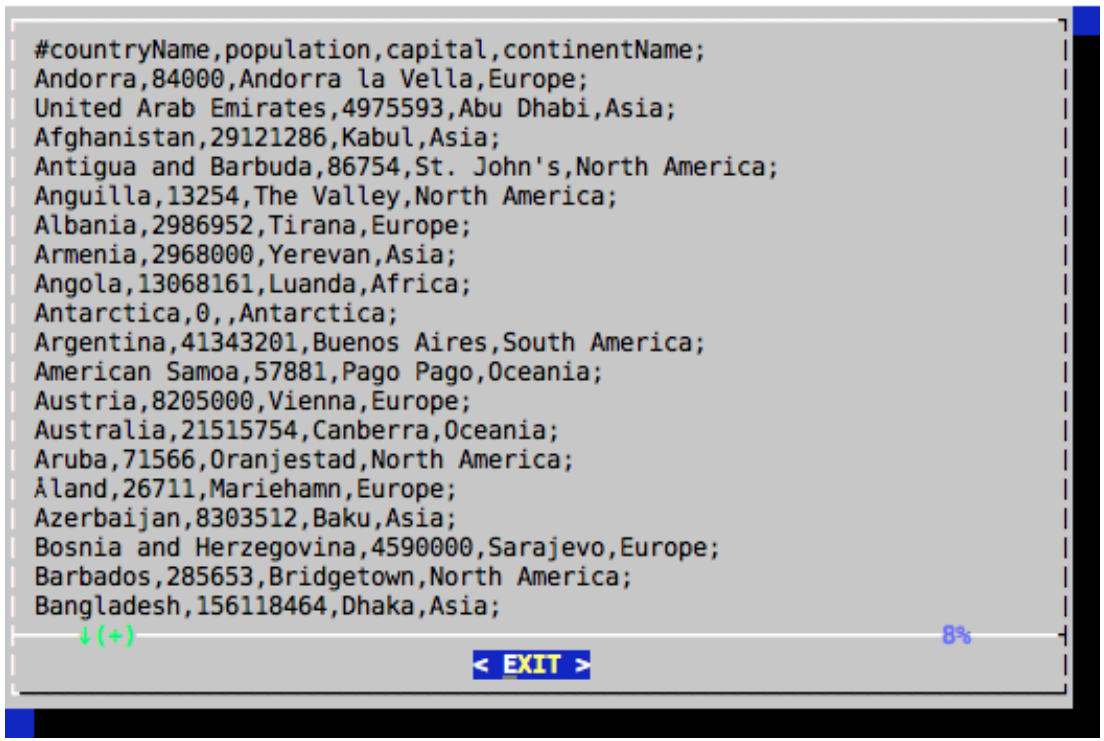
```
$ cat nameinput.sh
#!/bin/sh
name=$(cdialog --inputbox "Enter your name:" 0 0 3>&1 1>&2 2>&3)
cdialog --msgbox "Hello $name" 5 25
```

Using Textboxes to Show Content of Files

Displaying a lengthy text that exceeds the screen's capacity presents a challenge. For files with unpredictable content, determining the occupied space becomes impractical. Introduce scrollable text to the user using a `textbox`, similar to `more`/`less`. The lower right corner of the box indicates the percentage of displayed text. To showcase the content within the box, provide a file as an argument to the `textbox` option.

Example for Using textbox

```
$ cat showfile.sh
#!/bin/sh
cdialog --textbox $1 0 0
$ showfile.sh CapitalCities.txt
```

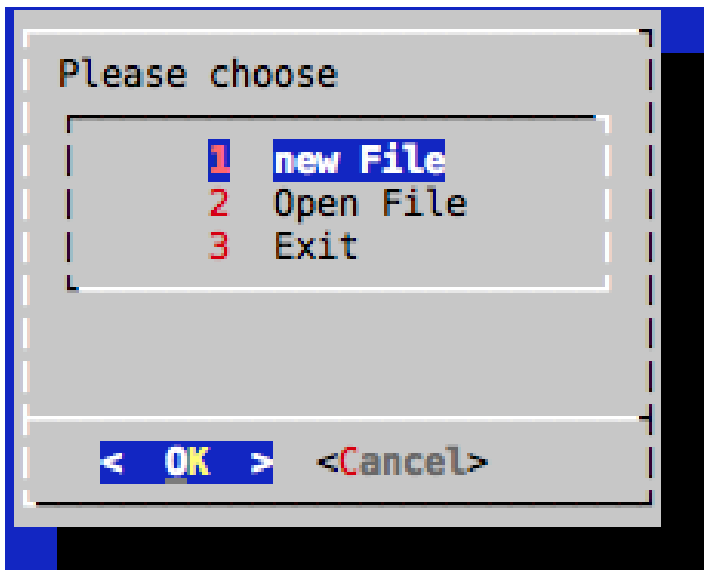


Menus with cdialog

When presenting a restricted set of choices to the user, a menu is often the appropriate selection. This is achieved using the `menu` option. The `menu` displays specific entries that can be internally set to different values as required. The practice of displaying strings to the user while storing numbers in variables (or vice versa) proves beneficial for both users and programmers.

Example Menu with cdialog

```
$ cat mainmenu.sh
#!/bin/sh
menuselection=$(cdialog --menu "Please choose" 0 0 0
"1" "New File" "2" "Open File" "3" "Exit" 3>&1 1>&2 2>&3)
$ mainmenu.sh
```



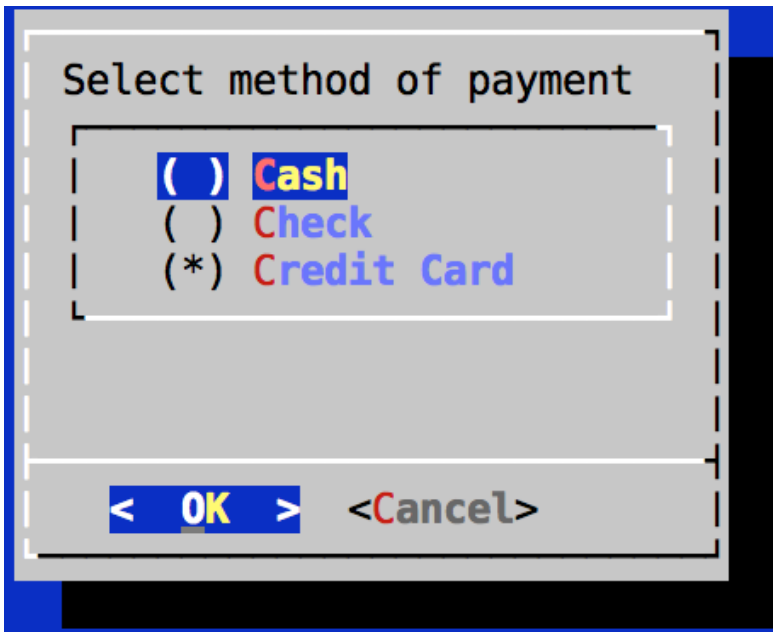

```
$ echo $?
2
```

Radiobutton-like Selection Using `radiolist`

Radiobuttons enable precise selection from a set of possibilities, akin to an old radio (hence the name). Pressing one button deselects other previously pressed buttons. Press the space key in `cdialog` to choose items. Buttons can default to either `off` or `on`.

Example Usage for radiolist

```
$ cat paymentmethod.sh
method=$(cdialog --radiolist "Select method of payment" 0 0 3 \
"Cash" "" off \
"Check" "" off \
"Credit Card" "" on 3>&1 1>&2 2>&3)
echo $method
```

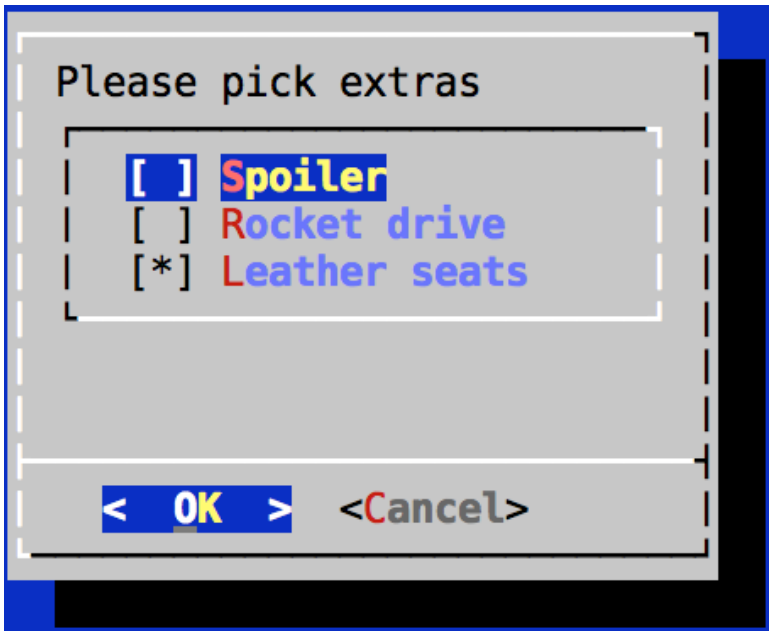


Selecting Entries with `checklist`

When a user needs the flexibility to select more than one entry, a `checklist` proves to be a good choice as it allows the selection of any number of entries. To apply this, duplicate a `radiolist` and replace it with a `checklist`.

Example for a checklist

```
$ cat extras.sh
#!/bin/sh
extras=$(cdialog --checklist "Please pick extras" 0 0 3 \
Spoiler "" off \
"Rocket drive" "" off \
"Leather seats" "" on 3>&1 1>&2 2>&3)
echo $extras
$ extras.sh
```



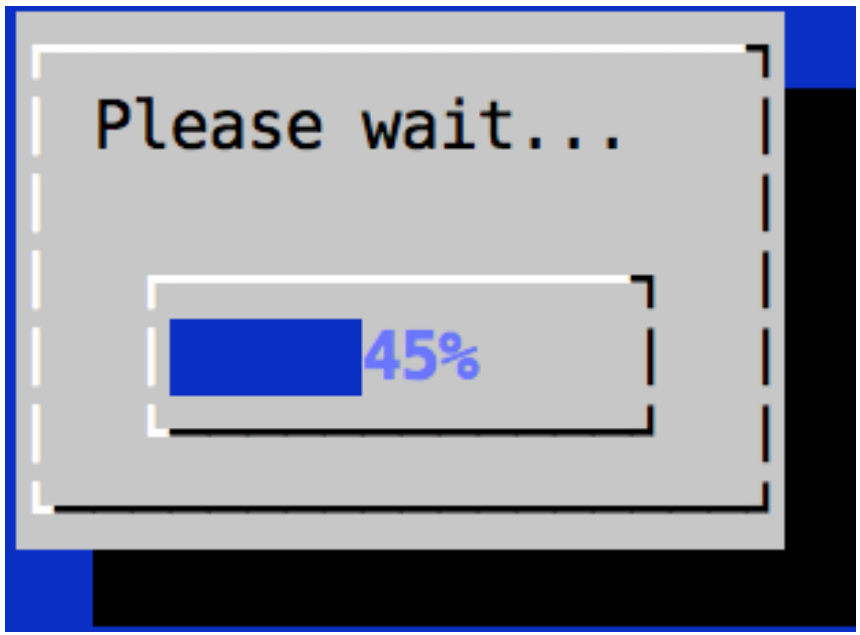

```
  Spoiler Leather seats
```

Progress Bars with gauge

Cdialog also supports progress bars through the gauge option, displaying an animated progression of an activity or operation. By passing numbers representing percentages to cdialog, you can visualize them as a progressing bar. Once there are no more numbers to process, the bar stops at the last value.

Example Progress Bar with gauge

```
$ cat progress.sh
#!/bin/sh
for p in 10 24 26 34 45 50 75 100;
do
    echo $p | cdialog --gauge "Please wait..." 8 30
    sleep 1;
done
```



Overview

1 Here documents

2 Text-based User Interfaces with `cdialog`

3 Functions

4 Traps

Functions in Shell Scripts

Shell functions offer the possibility to combine commands or shorten commands with parameters.
Don't confuse shell functions with shell programs:

- A shell program is a **file** that contains commands.
- The shell manages a shell function and remains in main memory, without retrieving it from disk.

The syntax for a shell function is as follows:

```
FunctionName() {
  Command1
  ...
  CommandN
}
```

Functions

A Shell function runs like a shell program. The parameters follow the name of the function. Using `unset FunctionName` removes the function.
Example: A function `usage` will display a helpful notice on how to use the program properly.

```
$ cat helpmessage.sh
#!/bin/sh

usage() {
    echo "Usage: $0 File"
    exit 1
}

if [ $# -lt 1 ]; then
    usage
fi
cat $1
```

Note: In bash and Korn shell prefix the keyword `function`. The C Shell does not support functions.

Passing Arguments to Functions

Functions can receive parameters without requiring explicit specification in the function declaration. These parameters are numbered similarly to the parameters passed to the shell script itself on the command line, mirroring the behavior of $1, $2, and so forth. Additionally, a special variable, $? , stores the return value of the function.

```
$ cat myparam.sh
#!/bin/sh

passmesomething() {
    echo "Here is what you passed: $1"
}

passmesomething "Pepper"
echo "The function returned $?"
$
$ sh ./myparam.sh
Here is what you passed: Pepper
The function returned 0
```

Note: Be careful when using `exit` within a shell function. When called, it will stop executing the whole script, not the function.

Local Variables in Functions

All variables within a script are globally accessible and are subject to global changes. To confine variables to a function's scope, prefix the keyword `local`. This designates variables as function-local, meaning they exist solely during the execution of the function and not before or after.

```
$ cat myglobal.sh
MYVAR=13

localfunc() {
    local MYVAR=2;
    echo "Within localfunc(): $MYVAR"
}
echo "Before localfunc(): $MYVAR"
localfunc $MYVAR
echo "After localfunc(): $MYVAR"
$ sh ./myglobal.sh
Before localfunc(): 13
Within localfunc(): 2
After localfunc(): 13
```

Without the `local` keyword, `$MYVAR` changes within `localfunc()` (global variable) and would still have that value when the function returns.

Shell Functions and return

Returning numbers from a function is not a problem, but strings are. Shell functions can not return strings that easily. There are three ways to solve this:

- Use a global string variable (outside the function) and set it to a new string in the function.
- echo the string to the screen and capture it in a variable with $?

```
function_that_returns_a_string() {
    ...
    echo $mystring
}
function_that_returns_a_string
mystringvar=$?
```

- A command substitution like this:

```
hello() {
    var="Hello world."
    echo "$var"
}

greeting=$(hello)
echo $greeting
```

Recursion in Shell Scripts

Recursive function calls are also possible. The keyword `return` leaves the function. Using the `break` keyword instead stops the loop or function prematurely (not used here).

```
1  $ cat factorial.sh
2  #!/bin/sh
3  factorial() {
4      if [ $1 -eq 1 ]; then
5          echo 1; return
6      else
7          echo $(expr $1 \* $$(factorial $(expr $1 - 1)))
8      fi
9  }
10 factorial $1
11 $ factorial 3
12 6
```

The recursion happens in line 7 where the function factorial() runs again with the current value of $1 minus one, using the returned value in the `expr` multiplication. Running the script with shell debug options shows the passed values. An exercise left for the reader...

Overview

1. Here documents
2. Text-based User Interfaces with `cdialog`
3. Functions
4. Traps


It's a
Trap!

Traps in Shell Scripts

Unix uses signals as a primitive form of communicating between processes (IPC). We already saw how to use the `kill` command to send signals to processes. The receiving processes can either:

- ignore the signal
- exit the process (more or less cleanly)
- execute a programmable signal handler routine

We create our custom signal handler routines in shell scripts. The built-in `trap` command allows shell scripts to capture signals and execute specific operations upon receiving them. A similar functionality is present in C through the `signal` function, with the `sigaction(2)` man page providing a list of signals from `signal.h` (system dependent).
The general syntax for a trap handler in shell is:

```
trap function signal [ signal ...  ]
```

For `signal`, use either the signal number or the signal name, but for portability reasons use the signal name.

Trap Example

This example traps two signals: `SIGINFO` and `SIGWINCH` (lines 15 and 16). Each call their own function: `info()` and `windowchange()`.

```
1  #!/bin/sh
2  ADJCOUNT=0
3  COUNT=0
4  info() {
5      echo "Seconds elapsed: $COUNT"
6      echo "Number of adjustments: $ADJCOUNT"
7  }
8
9  windowchange() {
10     echo "Window size changed."
11     ADJCOUNT=$(($ADJCOUNT + 1))
12 }
13
14 echo "Adjust your window size..."
15 trap info SIGINFO
16 trap windowchange SIGWINCH
17
18 while [ $COUNT -lt 60 ] ; do
19     COUNT=$(($COUNT + 1))
20     sleep 1
21 done
```

Example Output

```
sh ./mysignal.sh
Adjust your window size...
load: 1.86  cmd: sleep 39934 waiting 0.00u 0.00s
Seconds elapsed: 2
Number of adjustments: 0
load: 1.79  cmd: sleep 39937 waiting 0.00u 0.00s
Window size changed.
Seconds elapsed: 5
Number of adjustments: 1
```

Send `SIGINFO` to the program using the CTRL + T key-combo (see `stty -a` for mapped keys).

Further Information



[LSST] Vivek G. Gite
Linux Shell Scripting Tutorial - A Beginner's handbook
http://freeos.com/guides/lsst/index.html



[PSB] https://github.com/dylanaraps
pure sh bible
https://github.com/dylanaraps/pure-sh-bible



[SHPRJW] Jürgen Wolf
Shell-Programmierung - Einführung, Praxis, Referenz
http://openbook.galileocomputing.de/shell_programmierung



[AMDL] Apple Inc.
Mac Developer Library - Shell Scripting Primer
https://developer.apple.com/library/mac/documentation/OpenSource/Conceptual/ShellScripting/Introduction/Introduction.html